



Lesson 3

29.06.2026



```
public class Task1 {  
    public static void main(String[] args) {  
        byte a = 100;  
        char b = 'A';  
        System.out.println(a + b);  
    }  
}
```

```
public class Task2 {  
    public static void main(String[] args) {  
        double a = 9.7;  
        int b = (int) a;  
        System.out.println(b);  
    }  
}
```

```
public class Task3 {  
    public static void main(String[] args) {  
        Integer a = 100;  
        int b = a;  
        System.out.println(b);  
    }  
}
```

```
public class Task4 {  
    public static void main(String[] args) {  
        Integer a = 127;  
        Integer b = 127;  
        System.out.println(a == b);  
  
        Integer c = 256;  
        Integer d = 256;  
        System.out.println(c == d);  
    }  
}
```

Домашнє завдання: покрокова інструкція

1



Визначте, яку інформацію необхідно вивести на екран, і складіть **список необхідних полів**.

Приклад: інформація про студента

- ім'я (String)
- вік (int)
- середній бал (double)
- спеціальність (String)

Що потрібно зробити?

Подумайте, які дані ви хочете показати (наприклад, про студента, товар, книгу, працівника тощо) і запишіть їх у вигляді списку.

2



Створіть змінні з правильними типами даних (int, double, String тощо) залежно від того, які дані вони будуть зберігати.

```
// Оголошення змінних
String name;      // ім'я студента
int age;          // вік
double gpa;       // середній бал
String major;     // спеціальність
```

3



Заповніть змінні даними **першої записи** та виведіть всю інформацію на екран за допомогою `System.out.println()`.

```
// Дані першої записи
name = "Іван Петренко";
age = 20;
gpa = 4.35;
major = "Комп'ютерні науки";

System.out.println("Ім'я: " + name);
System.out.println("Вік: " + age);
System.out.println("Середній бал: " + gpa);
System.out.println("Спеціальність: " + major);
```

Результат (перша запис)

Ім'я: Іван Петренко
Вік: 20
Середній бал: 4.35
Спеціальність: Комп'ютерні науки

4



Первикористайте вже створені змінні, присвоївши їм значення **другої записи** (не створюючи нових змінних).

```
// Дані другої записи (ті самі змінні)
name = "Марія Коваленко";
age = 21;
gpa = 3.92;
major = "Інженерія програмного забезпечення";
```

Важливо!

Не створюйте нові змінні. Використовуйте ті самі змінні та змінійте лише їх значення.

5



Повторно виведіть інформацію на екран і переконайтесь, що відображаються дані вже **другої записи**.

```
// Вивід другої записи
System.out.println("Ім'я: " + name);
System.out.println("Вік: " + age);
System.out.println("Середній бал: " + gpa);
System.out.println("Спеціальність: " + major);
```

Результат (друга запис)

Ім'я: Марія Коваленко
Вік: 21
Середній бал: 3.92
Спеціальність: Інженерія програмного забезпечення



Порада: Використовуйте одні й ті самі змінні для різних даних. Просто присвоюйте нові значення та знову виводьте їх на екран.

Java Code Convention – Іменування

Єдині правила іменування роблять код читабельним, зрозумілим і підтримуваним.

Елемент	Правило іменування	Приклад
 Клас	PascalCase Перше слово з великої літери	CustomerService OrderProcessor
 Інтерфейс	PascalCase	PaymentGateway Runnable
 Enum (перелік)	PascalCase	OrderStatus UserRole
 Record	PascalCase	CustomerDto
 Метод	camelCase Починається з дієслова	calculateTotal() findUserById()
 Змінна (локальна)	camelCase	userName totalPrice
 Поле класу (атрибут)	camelCase	private String firstName; private int age;
 Константа (static final)	UPPER_SNAKE_CASE Всі літери великі, слова розділяються підкресленням	MAX_RETRY_COUNT DEFAULT_TIMEOUT
 Пакет	lowercase Всі літери малі	com.company.project.service
 Файл	Ім'я файлу збігається з іменем публічного класу	CustomerService.java

Ім'я файлу

Ім'я файлу повинно повністю збігатися з іменем публічного класу.

Код класу

```
public class OrderController {  
    // ...  
}
```

Ім'я файлу

OrderController.java

Як це виглядає у коді

```
package com.company.payment.service;  
  
public class PaymentService {  
    private static final int MAX_RETRY_COUNT = 3;  
    private String paymentId;    // поле класу (camelCase)  
  
    public void processPayment() {    // метод (camelCase)  
        String customerName = "John Doe";    // змінна (camelCase)  
        boolean isSuccess = true;    // булева змінна (camelCase)  
    }  
}
```

Булеві змінні

Назви булевих змінних бажано починати з:

is

has

can

should

Приклади:

- isActive
- hasPermission
- canEdit
- shouldRetry

Колекції

Назва повинна відображати множину об'єктів.

```
List<User> users;  
Set<Role> roles;  
Map<Long, Customer> customers;
```

Пакети

Завжди малими літерами.

- ✓ com.company.project
- ✓ com.bank.payment
- ✓ org.springframework.web
- ✗ Com.Company.Project



Швидка схема

Пакет – lowercase

Клас / Інтерфейс / Enum – PascalCase

Метод / Змінна / Поле – camelCase

Константа – UPPER_SNAKE_CASE

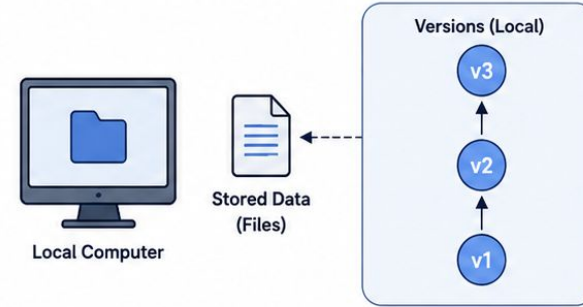
Файл – PascalCase.java



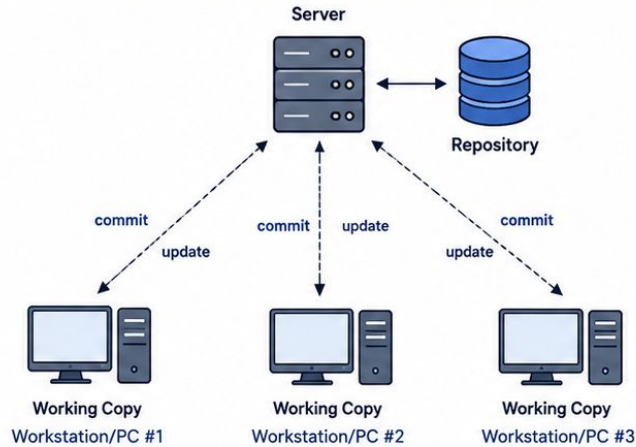
Types of Version Control System

- 1 Local Version Control System
- 2 Centralized Version Control System
- 3 Distributed Version Control System

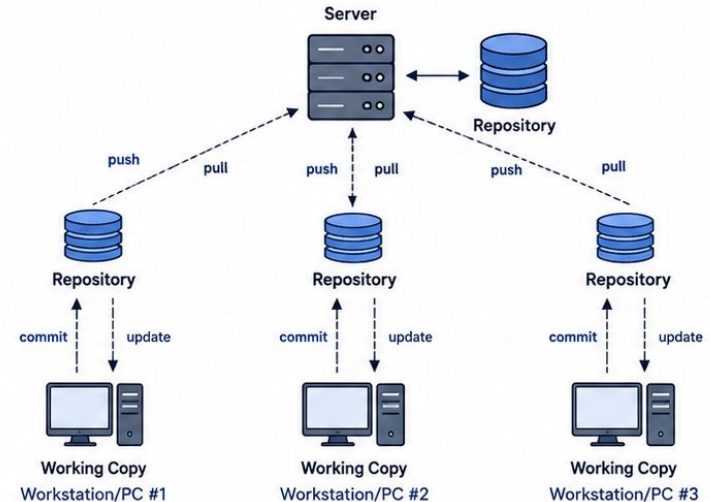
Local Version Control System



Centralized Version Control System



Distributed Version Control System

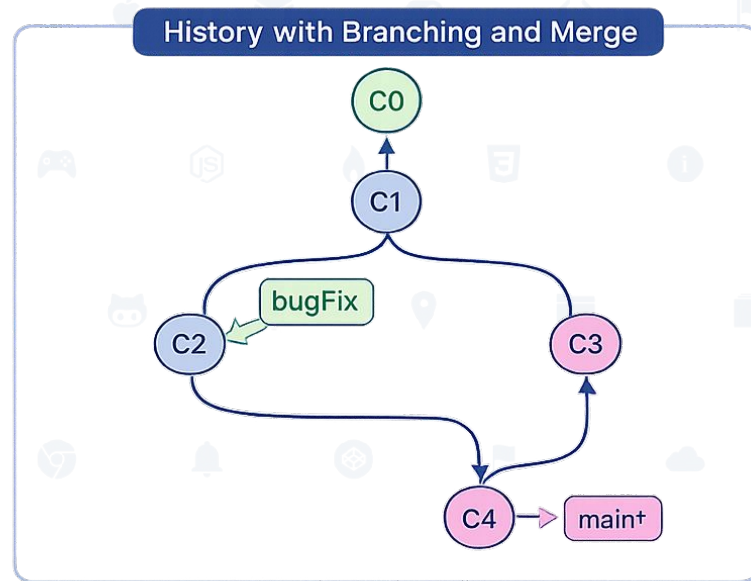
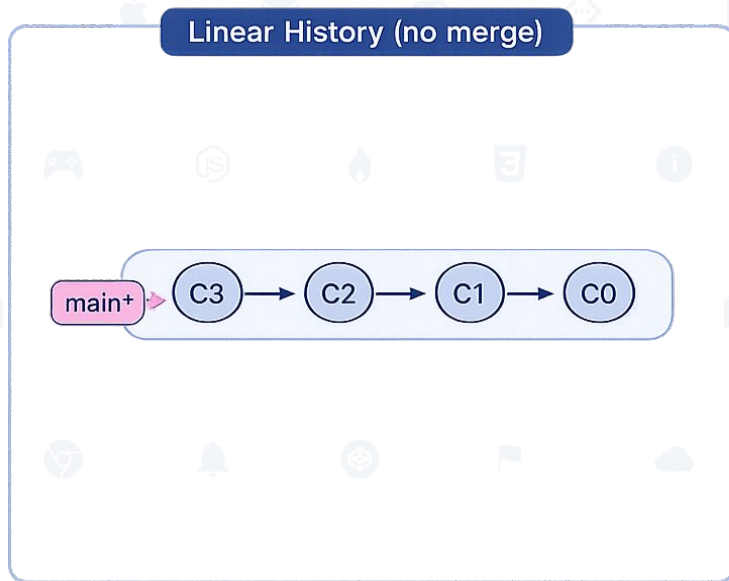


Git – розподілена система керування версіями. Проект був створений Лінусом Торвальдсом для управління розробкою ядра Linux, перша версія випущена 7 квітня 2005 року



Як працює

Якщо подивитися на картинку, то ставати трохи простіше з розумінням. Кожен гурток, це commit. Стрілки показують напрямок, з якого commit зроблений наступний. Наприклад C3 зроблено з C2 і т. д. Всі ці commit знаходяться у гілці під назвою main. Це основна гілка, найчастіше її називають master. Прямокутник main* показує в якому commit ми зараз знаходимося, вказівник.



Налаштування

Ви встановили собі Git та можете ним користуватися. Давайте тепер його налаштуємо, щоб, коли ви створювали commit, вказувався автор, хто його створив.

Відкриваємо термінал (Linux та MacOS) або консоль (Windows) та вводимо наступні команди.

```
#Установим имя для вашего пользователя
```

```
#Вместо <ваше_имя> можно ввести, например, Grisha_Popov
```

```
#Кавычки оставляем
```

```
git config --global user.name "<ваше_имя>"
```

```
#Теперь установим email. Принцип тот же.
```

```
git config --global user.email "<адрес_почты@email.com>"
```



Створення репозиторію

Тепер ви готові працювати з Git локально на комп'ютері.

Створимо наш перший репозиторій. Для цього пройдіть до папки вашого проекту.

```
#Для Linux и MacOS путь может выглядеть так /Users/UserName/Desktop/MyProject
```

```
#Для Windows например C://MyProject
```

```
cd <путь_к_вашему_проекту>
```

```
#Инициализация/создание репозитория
```

```
git init
```



Тепер Git відстежує зміни файлів вашого проекту. Але, тому що ви тільки створили репозиторій у ньому немає коду. Для цього потрібно створити commit.

#Добавим все файлы проекта в нам будущий commit

```
git add .
```

#Или так

```
git add --all
```

#Если хотим добавить конкретный файл то можно так

```
git add <имя_файла>
```

#Теперь создаем commit. Обязательно указываем комментарий.

#И не забываем про кавычки

```
git commit -m "<комментарий>"
```



Гілка – це набір commit, які йдуть один за одним. Гілка має назву, основну гілку найчастіше називають master / main. Якщо говорити простими словами, то гілка master / main – це наш проект. Інші гілки – це окреме місце для реалізації нового функціоналу або виправлення багів (помилки) нашого проекту. Тобто, з окремою гілкою ви робите будь-що, а потім зливаєте ці зміни в основну гілку master / main.

Не рекомендую створювати commit безпосередньо в master / main. Краще для цього заводити нову гілку та всі зміни писати там.

При створенні нової гілки, намагайтеся називати її коротким і ємним ім'ям. Щоб одразу було зрозуміло, що саме змінювалося за проектом. Якщо ви використовуєте якусь систему для ведення завдань, то можете на початку назви гілки вказувати ID завдання, щоб можна було легко знайти, на основі якого завдання було створено гілку.

Для того, щоб створити нову гілку, вводимо:

```
git branch <название_ветки>
```

#или вот так

```
git checkout -b <название_ветки>
```




Перемикається між гілками можна такою командою:

```
git checkout <название_ветки>
```

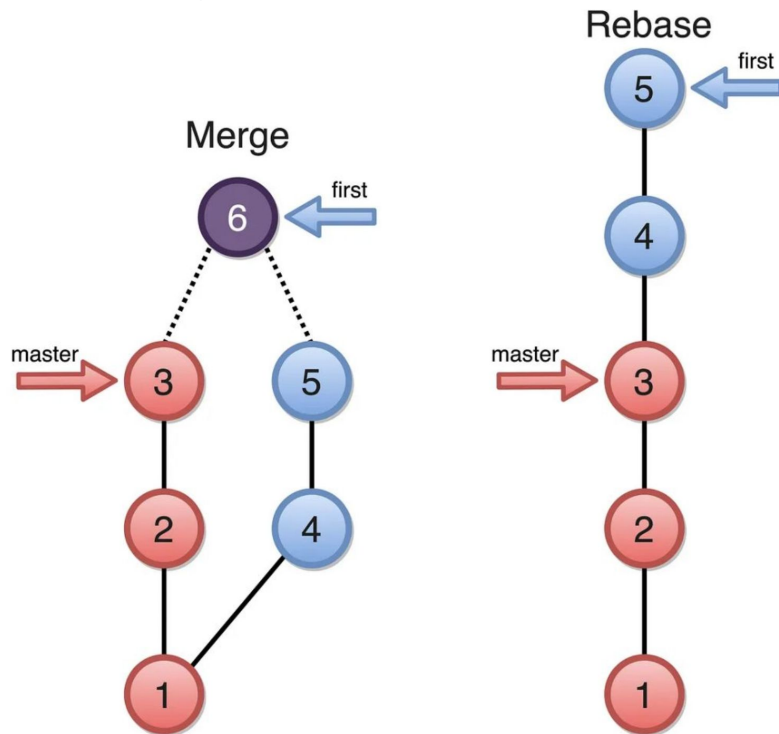
Після того, як ви завершили роботу над своїм завданням, гілку можна злити в master. Для цього потрібно перейти у гілку master і виконати наступну команду:

```
# Переключаемся в master
git checkout master
# Обновляем локальную ветку с сервера
git pull origin master

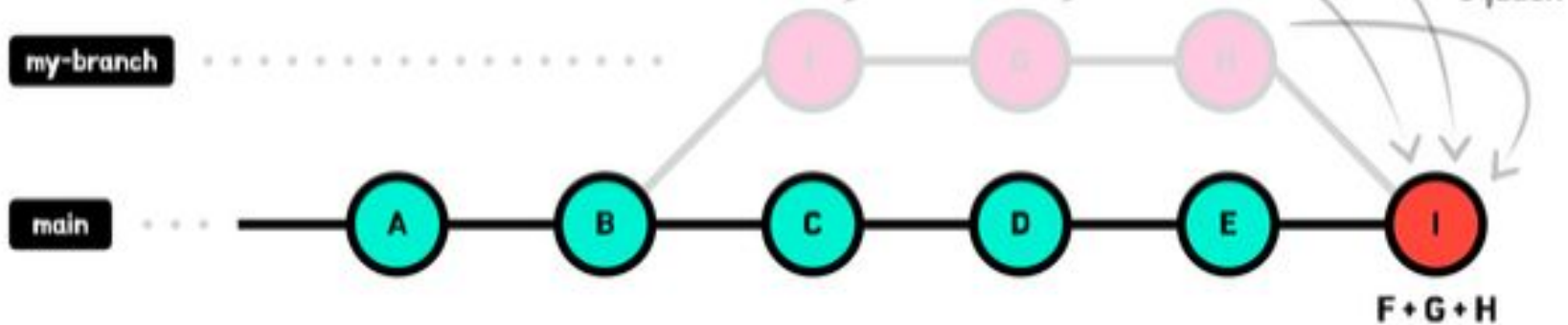
# Делаем merge вашей ветки, в ветку в которой вы находитесь
# В данном примере это master
git merge <название_ветки>
```



Rebase (перебазування) - один із способів у git, що дозволяє об'єднати зміни двох гілок. У цього способу є перевага перед merge (злиття) - він дозволяє переписати історію гілки, надавши той історії той вид, який нам потрібний.



Squash & Merge





Команда `git cherry-pick` бере зміни, що вносяться одним коммітом, і намагається повторно застосувати їх у вигляді нового комміту у поточній гілці. Ця можливість корисна в ситуації, коли потрібно забрати парочку коммітів з іншої гілки, а не зливати гілку повністю з усіма внесеними до неї змінами.





Команда **git commit --amend** - це зручний спосіб змінити останній коміт. Вона дозволяє об'єднати проіндексовані зміни із попереднім комітом без створення нового коміту. Її можна використовувати для редагування коментаря до попереднього коміту без зміни стану коду в ньому.

Команда **git squash** це прийом, який допомагає взяти серію комітів і ущільнити її. Наприклад, припустимо: у вас є серія з N комітів і ви можете шляхом стиснення перетворити її на один-єдиний коміт. Стиснення через git squash в основному застосовується, щоб перетворити велику кількість малозначимих комітів на невелике число значних. Так легше відстежувати історію Git.

Команда **git reset** – це складний універсальний інструмент для скасування змін. Вона має три основні форми виклику, що відповідають аргументам командного рядка --soft, --mixed, --hard. Кожен із цих трьох аргументів відповідає трьом внутрішнім механізмам керування станом Git: дереву комітів (HEAD), розділу проіндексованих файлів та робочого каталогу.

Команда **git stash** - ця команда використовується для збереження непідтверджених змін окреме сховище, щоб можна було повернутися до них пізніше. Самі файли повертаються до вихідного стану. Команда корисна, коли ви працюєте над однією гілкою, хочете перейти на іншу, але ви ще не готові зробити коміт у поточній гілці. Таким чином, ви ховаєте зміни в коді, перемикаєтеся на іншу гілку, повертаєтеся до вихідної гілки, а потім розархівуєте свої зміни.